STAD70 — LSTM Intraday Straddle Strategy

Feature Engineering, Return Calculation, and P&L Decomposition

Technical Reference: V13 Notebook

April 9, 2026

Contents

1 Overview and Data Inputs **2**

2 15-Minute Bar Resampling (`load_and_resample`) **2**

3 Straddle Series Construction (`build_straddle_series`) **3**

4 Feature Engineering (`build_features`) **3**

4.1 Auxiliary: Intraday Volatility Estimate σ_t (1 feature) . . . 3
4.2 Feature Group I: Normalised Returns (5 features) . . . 4
4.3 Feature Group II: MACD (3 features) . . . 5
4.4 Feature Group III: Momentum (4 features) . . . 5
4.5 Feature Group IV: Options-Specific Features (3 features) . . . 6
4.6 Feature Group V: SPY Realised Volatility (3 features) . . . 6
4.7 Complete Feature Vector Summary . . . 7

5 Return Calculation in the Objective Function (`compute_loss`) **7**

5.1 Overview of the Loss Function . . . 7
5.2 Step 1 — Vol-Scaled Return . . . 8
5.3 Step 2 — Turnover Cost . . . 8
5.4 Step 3 — Net Return, Sharpe, and ES . . . 8
5.5 Numerical Example . . . 8
5.6 Expected Shortfall Term . . . 9

6 P&L Calculation in the Backtest (`backtest`) **9**

6.1 Signal Timing and Position Alignment . . . 9
6.2 Vol-Scaling in the Backtest . . . 9
6.3 P&L Formula — Same-Strike Bars . . . 9
6.4 P&L Formula — Strike-Change Bars . . . 9
6.5 Trading Table (Mapping to Requested Columns) . . . 10
6.6 Step-by-Step Walkthrough (Based Solely on the Trading Table Above) . . . 10
6.7 Summary: Loss Function vs. Backtest P&L . . . 11

1 Overview and Data Inputs

The strategy adapts Tan, Roberts & Zohren (ICAIF 2024) to intraday 15-minute bars on 0-DTE SPY options. Two raw data sources are consumed:

- **SPY ETF** (1-min bars): `timestamp, open, high, low, close, volume, vwap, transactions`
- **SPY options** (1-min bars): `timestamp, type (C/P), strike, open, high, low, close, volume, vwap`

Global constants relevant throughout this document:

$$\text{BAR_MINUTES} = 15, \quad \text{BARS_PER_DAY} = \left\lfloor \frac{6.5 \times 60}{15} \right\rfloor = 26, \quad \mathcal{A} = \sqrt{252 \times 26} \approx 81.03 \text{ (annualisation factor)}$$

$$\sigma^{\text{tgt}} = 0.15, \quad c = 3 \text{ bps} = 3 \times 10^{-4}, \quad \tau = 20 \text{ (LSTM window)}, \quad \text{VOL_EWM_SPAN} = 26$$

2 15-Minute Bar Resampling (`load_and_resample`)

Each 15-minute bar is formed from the 15 one-minute bars with label "left, closed left", so the bar labelled 09:30 covers [09:30, 09:45). Aggregation rules differ by series:

Series	Field	Aggregation
SPY ETF	open	first
	high	max
	low	min
	close	last
	volume	sum
	vwap	**mean of 1-min vwaps**
Options	open	first
	close	last
	volume	sum

Note on options VWAP. The `load_and_resample` function does *not* aggregate the options `vwap` column. When a VWAP-based execution price is required for the trading log (Section 6.5), it must be looked up directly from the original 1-minute options data as the mean of the per-minute VWAPs within the 15-minute window — it is *not* available as a column in `opt_bars`.

Only regular trading hours [09:30, 15:45] are retained, giving at most 26 bars per day.

Numerical Example — 15-min bar formation (SPY ETF)

1-min bar	open	high	low	close	volume	vwap
09:30	384.00	384.10	383.95	384.05	644,288	384.02
09:31	384.05	384.20	384.00	384.15	329,484	384.10
...			... (13 more 1-min bars) ...			
09:44	384.30	384.50	384.25	384.40	195,021	384.38
15-min bar 09:30	**384.00**	**384.55**	**383.90**	**384.40**	**5,847,232**	**384.22**

The 15-min bar VWAP = 384.22 is the arithmetic mean of the 15 one-minute VWAPs. For options, the equivalent would be the mean of the per-minute VWAPs for each `(type, strike)` group within the same 15-minute window, looked up from the raw 1-minute data.

3 Straddle Series Construction (`build_straddle_series`)

For each 15-minute bar t:

1. Find all strikes with *both* a call and a put quoted.
2. Select the ATM strike using the bar's *open* price (no leakage): $K^* = \arg\min_K |K - S_t^{\text{open}}|$, subject to $0.95 \le S_t^{\text{open}}/K^* \le 1.05$.
3. Record straddle prices: $V_t^{\text{op}} = C_t^{\text{op}} + P_t^{\text{op}}, \quad V_t^{\text{cl}} = C_t^{\text{cl}} + P_t^{\text{cl}}$.
4. Compute the log-return:

$$r_t^{\text{str}} = \begin{cases} \ln\!\left(V_t^{\text{cl}}/V_{t-1}^{\text{cl}}\right) & \text{if } K_t^* = K_{t-1}^* \text{ (same strike, same day)} \\ \ln\!\left(V_t^{\text{cl}}/V_t^{\text{op}}\right) & \text{if strike changed or first bar of day} \end{cases}$$

5. Drop rows where r_t^{str} is NaN (first bar of each day).

Numerical Example — Straddle series

Bar	Time	K^*	C^{op}	C^{cl}	P^{op}	P^{cl}	V^{op}	V^{cl}	r_t^{str}
t_0	09:30 Jan 3	384	2.60	2.50	2.40	2.50	5.00	5.00	NaN (dropped)
t_1	09:45 Jan 3	384	2.60	2.72	2.40	2.23	5.00	4.95	$\ln(4.95/5.00) = -0.01005$
t_2	10:00 Jan 3	384	2.72	2.80	2.23	2.35	4.95	5.15	$\ln(5.15/4.95) = +0.03922$
t_3	10:15 Jan 3	384	2.80	2.70	2.35	2.30	5.15	5.00	$\ln(5.00/5.15) = -0.02985$
t_4	09:30 Jan 4	385	2.90	2.80	2.80	2.75	5.70	5.55	NaN (dropped)
t_5	09:45 Jan 4	385	2.80	2.90	2.75	2.65	5.55	5.55	$\ln(5.55/5.55) = 0.00000$
t_6	10:00 Jan 4	385	2.90	2.95	2.65	2.63	5.55	5.58	$\ln(5.58/5.55) = +0.00540$
t_7	10:15 Jan 4	385	2.95	2.82	2.63	2.68	5.58	5.50	$\ln(5.50/5.58) = -0.01433$

The resulting `feat_df` (after `dropna`) contains rows $t_1, t_2, t_3, t_5, t_6, t_7$.

4 Feature Engineering (`build_features`)

All features at bar t use *only* data from bars $\le t-1$ (via `.shift(1)` inside rolling/EWM operations). The `FEATURE_COLS` list has $d = \mathbf{19}$ dimensions:

Group	Column names	Description	# cols
Aux	`sigma_t`	Intraday EWMSD of straddle log-return	1
I	`norm_ret_{1,4,8,13,26}`	Normalised return over k bars	5
II	`macd_{2_8,4_16,8_32}`	MACD signal (Baz et al. 2015)	3
III	`mom_{1,5,10,20}`	Rolling mean return over n bars	4
IV	`log_moneyness_call, _put, dte_frac`	Options-specific features	3
V	`spy_vol_{5,10,20}`	SPY realised vol over w bars	3
	Total		**19**

4.1 Auxiliary: Intraday Volatility Estimate σ_t (1 feature)

Formula:

$$\sigma_t^{\text{raw}} = \text{EWMSD}\!\left(r_{1:t-1}^{\text{str}} \;\middle|\; \text{same day, span} = 26\right)\Big|_{\text{shifted by 1 bar}}$$

Implementation (`build_features`, three lines):

1. Within each trading day, compute a cumulative EWM standard deviation of straddle log-returns using span = 26, then shift the result forward by 1 bar so that bar t's value is estimated from returns *before* bar t.

2. `sigma_t_filled = sigma_t_raw.fillna(sigma_t_raw.expanding().median())` — any NaN entry (e.g. the first few bars of a day before two observations are available) is replaced by the expanding median of all *valid* EWMSD values seen so far across the entire dataset.
3. `.where(sigma_t_filled.notna(), other=1e-4).clip(lower=1e-6)` — if the expanding median itself is still NaN (i.e. no valid EWMSD value has been produced yet), the hard fallback **1e-4** is used.

What does the filled value look like in practice? In a dataset with hundreds of trading days the expanding median stabilises quickly (within the first few weeks) at a value representing the typical intraday straddle volatility — empirically around 0.01–0.03 per bar for SPY 0-DTE options. Once the real EWMSD values become available (after the second bar of each day), σ_t is driven by those live estimates rather than the fallback.

Why 1e-4 as the last-resort fallback? If the very first bar of the entire dataset is being processed, there is no prior EWMSD estimate and no expanding median, so the code uses 10^{-4}. This implies a very large vol-scaling factor ($\sigma^{\text{tgt}}/\sigma_t = 0.15/0.0001 = 1500$), which is then hard-capped at 20.0 by the backtest: `vol_scale = (SIGMA_TGT / sigma_t.clip(lower=1e-4)).clip(upper=20.0)`.

Numerical example (toy 6-bar dataset):

With only 6 bars, no within-day EWMSD becomes available (span= 26 requires more than 2 observations before producing a finite standard deviation, and each day has only 3 bars here). Consequently σ_t^{raw} is NaN for every row; the expanding median is also NaN; and the hard fallback applies: $\sigma_t = 10^{-4}$ for all rows. In the tables below we use the representative value $\sigma_t = 0.0200$ — corresponding to a typical live-run estimate — so that the P&L numbers are interpretable. This is clearly labelled as an *assumed* value whenever it appears.

Bar	Time	r_t^{str}	EWMSD (raw, within day, up to t)	σ_t^{raw} (shifted, used at t)	Effective σ_t
t_1	09:45 Jan 3	−0.01005	NaN	NaN	*0.0200* (assumed)
t_2	10:00 Jan 3	+0.03922	NaN (< 2 obs)	NaN	*0.0200* (assumed)
t_3	10:15 Jan 3	−0.02985	≈ 0.0352	NaN	*0.0200* (assumed)
t_5	09:45 Jan 4	0.00000	NaN (new day)	NaN	*0.0200* (assumed)
t_6	10:00 Jan 4	+0.00540	NaN (< 2 obs)	NaN	*0.0200* (assumed)
t_7	10:15 Jan 4	−0.01433	≈ 0.0038	NaN	*0.0200* (assumed)

4.2 Feature Group I: Normalised Returns (5 features)

Lookbacks: $k \in \{1, 4, 8, 13, 26\}$ bars.

Formula:

$$\text{norm_ret}_k(t) = \left. \frac{\sum_{j=1}^{k} r_{t-j}^{\text{str}}}{\sigma_t \sqrt{k}} \right|_{\text{clip}[-10,10]}$$

The numerator is the rolling sum of the k log-returns strictly before bar t, computed as `ret.shift(1).rolling(k, min_periods=1).sum()`. Division by $\sigma_t\sqrt{k}$ standardises to an approximately unit-normal scale.

Numerical example (using $\sigma_t = 0.0200$ throughout):

Bar	Time	Past k=1 return	**norm_ret_1**	**norm_ret_4**	Notes
t_1	09:45 Jan 3	NaN $\to 0$	$0 \,/\, (0.020 \cdot 1) = 0.000$	0	No prior bar
t_2	10:00 Jan 3	$r_{t_1} = -0.01005$	$-0.01005/0.020 = -0.503$	$\frac{0-0.01005}{0.020\sqrt{4}} = -0.251$	
t_3	10:15 Jan 3	$r_{t_2} = +0.03922$	$+0.03922/0.020 = +1.961$	$\frac{0-0.01005+0.03922}{0.020\cdot 2} = +0.729$	2 valid returns
t_5	09:45 Jan 4	NaN $\to 0$	0.000	0	New day, NaN$\to$0
t_6	10:00 Jan 4	$r_{t_5} = 0.00000$	0.000	0	
t_7	10:15 Jan 4	$r_{t_6} = +0.00540$	$+0.00540/0.020 = +0.270$	$+0.270/\sqrt{4} = +0.135$	

4.3 Feature Group II: MACD (3 features)

Pairs: $(S, L) \in \{(2, 8), (4, 16), (8, 32)\}$ bars, using `zip` (not `itertools.product`; the latter would give 9 pairs instead of 3).

Half-life conversion (Baz et al. 2015): $h = \ln(0.5)/\ln(1 - 1/\text{span})$, e.g. $S = 2 \Rightarrow h_S = 1.0$; $L = 8 \Rightarrow h_L \approx 5.19$.

Construction steps:

$$\text{EMA}_S(t) = \text{EWMA}(V^{\text{cl}}, h_S)\big|_{\text{shift}(1)}, \qquad \text{EMA}_L(t) = \text{EWMA}(V^{\text{cl}}, h_L)\big|_{\text{shift}(1)}$$

$$\text{MACD_raw}(t) = \big(\text{EMA}_S - \text{EMA}_L\big)/\text{price_std5} \quad (\text{price_std5: rolling 5-bar std of close, shifted by 1})$$

$$\text{MACD_norm}(t) = \text{MACD_raw}/\text{rolling 20-bar std of MACD_raw}\,\Big|_{\text{clip}[-3,3]}$$

$$\text{macd}_{S,L}(t) = \phi\big(\text{MACD_norm}\big), \qquad \phi(y) = \frac{y\, e^{-y^2/4}}{0.89} \ (\text{Baz et al.})$$

The ϕ transform assigns near-zero weight to extreme values ($|y| \gg 1$), preventing large MACD readings from dominating the signal.

4.4 Feature Group III: Momentum (4 features)

Lookbacks: $n \in \{1, 5, 10, 20\}$ bars.

Formula:

$$\text{mom}_n(t) = \frac{1}{n}\sum_{j=1}^{n} r^{\text{str}}_{t-j} = \text{RollingMean}\big(r^{\text{str}},\, n,\, \text{min_periods} = 1\big)\big|_{\text{shift}(1)}$$

Unlike Feature I, there is no normalisation by σ_t or $\sqrt{n}$ — this measures raw directional momentum.

Bar	Time	r^{str}_{t-1}	**mom_1**	**mom_5**	**mom_20**
t_1	09:45 Jan 3	NaN$\to$0	0.000	0.000	0.000
t_2	10:00 Jan 3	-0.01005	-0.01005	-0.00503	-0.00503
t_3	10:15 Jan 3	$+0.03922$	$+0.03922$	$+0.01459$	$+0.01459$
t_5	09:45 Jan 4	$r_{t_3} = -0.02985^{\dagger}$	-0.02985	$-0.00214^{\ddagger}$	-0.00214
t_6	10:00 Jan 4	$r_{t_5} = 0.00000$	0.000	$+0.00174$	$+0.00174$
t_7	10:15 Jan 4	$r_{t_6} = +0.00540$	$+0.00540$	$+0.00294$	$+0.00294$

† The rolling window does *not* reset at day boundaries (no `groupby date`). Bar t_5 therefore includes r_{t_3} from Jan 3 in its lookback — a mild cross-day contamination that is more pronounced for larger n.

‡ mom_5 at t_5: mean of $(r_{t_1}, r_{t_2}, r_{t_3}, \text{NaN}, r_{t_5\ \text{shifted}}) = (-0.01005 + 0.03922 - 0.02985)/3 = -0.00068/3...$ using 4 available values: $(-0.01005 + 0.03922 - 0.02985 + 0)/4 = -0.00017$. (Exact value depends on `min_periods` treatment of NaN; shown here as representative.)

4.5 Feature Group IV: Options-Specific Features (3 features)

These features are structural properties of the bar, computed from the straddle-series columns (no additional shifting required):

$$\text{log_moneyness_call}(t) = \ln\!\big(S_t^{\text{op}}/K_t^*\big), \qquad \text{log_moneyness_put}(t) = \ln\!\big(K_t^*/S_t^{\text{op}}\big)$$

$$\text{dte_frac}(t) = \frac{\max(\text{minutes to 15:59},\ 1)}{252 \times 390} \qquad (252 \times 390 = 98{,}080 \text{ min/trading year})$$

Bar	Time	S^{op}	K^*	**log_m_call**	**log_m_put**	mins left	**dte_frac**
t_1	09:45 Jan 3	384.30	384	+0.000781	−0.000781	374	0.003813
t_2	10:00 Jan 3	383.80	384	−0.000521	+0.000521	359	0.003660
t_3	10:15 Jan 3	384.50	384	+0.001301	−0.001301	344	0.003507
t_5	09:45 Jan 4	385.20	385	+0.000519	−0.000519	374	0.003813
t_6	10:00 Jan 4	384.80	385	−0.000519	+0.000519	359	0.003660
t_7	10:15 Jan 4	385.10	385	+0.000260	−0.000260	344	0.003507

4.6 Feature Group V: SPY Realised Volatility (3 features)

Windows: $w \in \{5, 10, 20\}$ bars.

Formula:

$$\text{spy_vol}_w(t) = \text{std}\big(\{r_{t-w}^{\text{SPY}}, \ldots, r_{t-1}^{\text{SPY}}\}\big), \qquad r_t^{\text{SPY}} = \ln\!\big(S_t^{\text{cl}}/S_t^{\text{op}}\big)$$

Does it reset at day boundaries? No. The implementation is:

```
df["spy_log_ret"].shift(1).rolling(w, min_periods=1).std()
```

There is *no* `.groupby("date")`, so the rolling window slides over the full `feat_df` without resetting at the start of each trading day. A bar early in day $d+1$ will therefore include SPY returns from day d in its lookback window — a mild cross-day contamination that grows with window size w.

Numerical example. Define SPY intrabar log-returns $r_t^{\text{SPY}} = \ln(S_t^{\text{cl}}/S_t^{\text{op}})$:

Bar	Time	S^{op}	S^{cl}	r_t^{SPY}
t_0	09:30 Jan 3	384.00	384.40	+0.001041
t_1	09:45 Jan 3	384.30	384.20	−0.000260
t_2	10:00 Jan 3	383.80	384.00	+0.000521
t_3	10:15 Jan 3	384.50	384.50	+0.000000
t_4	09:30 Jan 4	385.00	385.20	+0.000519 (dropped from `feat_df`)
t_5	09:45 Jan 4	385.20	385.10	−0.000260
t_6	10:00 Jan 4	384.80	385.00	+0.000520
t_7	10:15 Jan 4	385.10	385.00	−0.000260

After `.shift(1)`, the values available in `feat_df` (rows t_1–t_7, ordered as rows 0–5) are:

Row	Timestamp	Shifted-by-1 spy ret	Source
0	t_1	NaN	shifted from t_0 which is not in `feat_df`*
1	t_2	$r_{t_1}^{\text{SPY}} = -0.000260$	
2	t_3	$r_{t_2}^{\text{SPY}} = +0.000521$	
3	t_5	$r_{t_3}^{\text{SPY}} = +0.000000$	cross-day (Jan 3 → Jan 4)
4	t_6	$r_{t_5}^{\text{SPY}} = -0.000260$	
5	t_7	$r_{t_6}^{\text{SPY}} = +0.000520$	

* t_0 is dropped from `feat_df` (first bar of day); the shifted value at row 0 is therefore NaN.

Computing **spy_vol_5** (rolling 5-bar std, `ddof=1`, `min_periods=1`):

Row	Timestamp	Values in rolling window (NaN dropped)	**spy_vol_5**
0	t_1	{} (only NaN)	0.000 (filled)
1	t_2	$\{-0.000260\}$ (single value, ddof=1: NaN)	0.000 (filled)
2	t_3	$\{-0.000260,\ +0.000521\}$	$s_2 = \sqrt{\frac{(0.000260-\bar{x})^2+(0.000521-\bar{x})^2}{1}}$
		$\bar{x} = 0.0001305$; deviations: $-0.0004905, +0.0003905$	$= \sqrt{4.316 \times 10^{-7}} \approx 0.000657$
3	t_5	$\{-0.000260,\ +0.000521,\ 0.000\}$	$\bar{x} = 0.0000870$; 3 values
		dev: $-0.000347,\ +0.000434,\ -0.000087$	$= \sqrt{3.076 \times 10^{-7}} \approx 0.000555$
4	t_6	$\{-0.000260,\ +0.000521,\ 0.0,\ -0.000260\}$	$\bar{x} \approx 0$; 4 values ≈ 0.000369
5	t_7	$\{-0.000260, +0.000521, 0.0, -0.000260, +0.000520\}$	5 values ≈ 0.000390

spy_vol_10 and **spy_vol_20** follow the same formula with $w = 10$ and $w = 20$; in this 6-bar example they are numerically identical to **spy_vol_5** since fewer than 10 bars are available.

4.7 Complete Feature Vector Summary

Index	Feature column	Group	d
1	`sigma_t`	Aux	1
2	`norm_ret_1`	I	
3	`norm_ret_4`	I	
4	`norm_ret_8`	I	
5	`norm_ret_13`	I	
6	`norm_ret_26`	I	5
7	`macd_2_8`	II	
8	`macd_4_16`	II	
9	`macd_8_32`	II	3
10	`mom_1`	III	
11	`mom_5`	III	
12	`mom_10`	III	
13	`mom_20`	III	4
14	`log_moneyness_call`	IV	
15	`log_moneyness_put`	IV	
16	`dte_frac`	IV	3
17	`spy_vol_5`	V	
18	`spy_vol_10`	V	
19	`spy_vol_20`	V	3
	Total		**19**

5 Return Calculation in the Objective Function (`compute_loss`)

5.1 Overview of the Loss Function

The LSTM is trained to maximise a combined objective:

$$\mathcal{L} = \underbrace{-\widehat{\mathrm{SR}}}_{\text{maximise Sharpe}} + \underbrace{\lambda_{\mathrm{ES}} \cdot \mathrm{ES}_\alpha}_{\text{penalise tail risk}} - \underbrace{\lambda_{\mathrm{PnL}} \cdot \bar{R}^{\mathrm{net}}}_{\text{optional: direct return}}$$

$$\widehat{\mathrm{SR}} = \frac{\bar{R}^{\mathrm{net}}}{\hat{\sigma}_{R^{\mathrm{net}}}} \cdot \mathcal{A}$$

The third term, $-\lambda_{\mathrm{PnL}} \cdot \bar{R}^{\mathrm{net}}$, directly maximises the mean net return without any volatility adjustment. It is controlled by the `pnl_lambda` argument (default = 0.0, meaning the term is absent unless explicitly enabled). When $\lambda_{\mathrm{PnL}} > 0$, the model balances Sharpe maximisation against raw P&L maximisation, which can bias it toward high-return but potentially volatile strategies.

5.2 Step 1 — Vol-Scaled Return

Given the LSTM output signal $X_t \in (-1, 1)$ and current intraday volatility σ_t:

$$R_t = X_t \cdot \underbrace{\frac{\sigma^{\text{tgt}}}{\sigma_t}}_{\text{vol scale}} \cdot r_t^{\text{str}}$$

The vol-scaling factor targets a constant annualised volatility of $\sigma^{\text{tgt}} = 15\%$: when σ_t is small (calm market) the position is scaled up; when σ_t is large (turbulent market) it is scaled down.

5.3 Step 2 — Turnover Cost

The paper's Eq. (13) deducts a cost proportional to the change in vol-scaled position. The V7 fix uses σ_{t-1} for the *previous* position:

$$\Delta_t = \left| \frac{X_t}{\sigma_t} - \frac{X_{t-1}}{\sigma_{t-1}} \right|, \qquad \text{TC}_t = c \cdot \sigma^{\text{tgt}} \cdot \Delta_t \qquad (c = 3 \times 10^{-4})$$

σ_{t-1} is obtained by shifting σ_t one step inside the trajectory: `sigma_prev = torch.cat([sigma_t[:,:1], sigma_t[:,:-1]], dim=1)`.

5.4 Step 3 — Net Return, Sharpe, and ES

$$R_t^{\text{net}} = R_t - \text{TC}_t$$

$$\widehat{\text{SR}} = \frac{\overline{R^{\text{net}}}}{\text{std}(R^{\text{net}})} \cdot \mathcal{A}, \qquad \text{ES}_\alpha = \frac{1}{k} \sum_{i=1}^{k} (-R^{\text{net}})_{(i)}, \quad k = \max\big(1, \lfloor \alpha N \rfloor\big)$$

where $(\cdot)_{(i)}$ is the i-th order statistic of $-R^{\text{net}}$ (worst returns first).

5.5 Numerical Example

We trace a single trajectory window of $\tau^{\text{ex}} = 4$ steps (a self-contained illustrative sequence; the deployed model uses $\tau = 20$). All $\sigma_t = 0.0200$ (assumed representative value; see Section 4.1), $\sigma^{\text{tgt}} = 0.15$, $c = 3 \times 10^{-4}$.

Step	t	X_t	σ_t	r_t^{str}	R_t	X_{t-1}	σ_{t-1}	Δ_t	TC_t
1	t_1	0.00	0.0200	−0.01005	0.0000	0.00⋆	0.0200⋆	0.00	0.000000
2	t_2	+0.60	0.0200	+0.03922	+0.17649	0.00	0.0200	30.00	0.001350
3	t_3	+0.70	0.0200	−0.02985	−0.15683	+0.60	0.0200	5.00	0.000225
4	t_5	0.00	0.0200	+0.00000	0.0000	+0.70	0.0200	35.00	0.001575

⋆ Step 1 uses $X_0 = 0$ (previous signal initialised to zero at start of trajectory).

Verification of Step 2:

$$R_{t_2} = 0.60 \times (0.15/0.0200) \times 0.03922 = 0.60 \times 7.5 \times 0.03922 = +0.17649$$
$$\Delta_{t_2} = |0.60/0.0200 - 0.00/0.0200| = |30 - 0| = 30$$
$$\text{TC}_{t_2} = 3 \times 10^{-4} \times 0.15 \times 30 = 0.001350$$

Step	R_t	TC_t	R_t^{net}
1	+0.000000	0.000000	+0.000000
2	+0.176490	0.001350	+0.175140
3	−0.156830	0.000225	−0.157055
4	+0.000000	0.001575	−0.001575
$\bar{R}^{\text{net}}$			+0.004128
$\hat{\sigma}_{R^{\text{net}}}$ (ddof=1)			0.14609
$\widehat{\text{SR}} = 0.004128/0.14609 \times 81.03$			**+2.288**

5.6 Expected Shortfall Term

After flattening the R_t^{net} values from all steps in the batch into a single vector `flat` of length N:

$$k = \max\big(1,\ \lfloor \alpha \cdot N \rfloor\big), \qquad \text{ES}_\alpha = \frac{1}{k}\sum_{i=1}^{k}(-R^{\text{net}})_{(i)}$$

In the illustrative trajectory of $\tau^{\text{ex}} = 4$ steps above, $N = 4$ and $\alpha = 0.05$, giving $k = \max(1, \lfloor 0.05 \times 4 \rfloor) = \max(1, 0) = 1$. The single worst return is $R_{t_3}^{\text{net}} = -0.157055$, so: $\text{ES}_{0.05} = 0.157055$.

Total loss $= -2.288 + 0.1 \times 0.157055 - 0 \times 0.004128 = -2.272$.

Note: in the full training run with $\tau = 20$ bars per window and a large batch, $N \gg 4$ so $k = \lfloor 0.05N \rfloor \gg 1$ and the ES averages over many tail observations.

6 P&L Calculation in the Backtest (`backtest`)

6.1 Signal Timing and Position Alignment

The position *held during* bar t equals the signal decided at bar $t-1$:

$$\text{held_pos}_t = X_{t-1} \qquad (\texttt{signal.shift(1)})$$

Additional constraints:

- **First bar of each day**: signal forced to 0 (no overnight carry-over).
- **Last bar of each day**: signal forced to 0 (mandatory EOD close).

6.2 Vol-Scaling in the Backtest

$$\text{vol_scale}_t = \min\Big(\sigma^{\text{tgt}}/\sigma_t,\ 20\Big), \quad \sigma_t \text{ clipped to } [10^{-4}, \infty)$$

Using $\sigma_t = 0.0200$ (assumed): vol_scale $= 0.15/0.020 = 7.5$.

6.3 P&L Formula — Same-Strike Bars

$$\text{straddle_pnl}_t = \text{held_pos}_t \times \text{vol_scale}_t \times \underbrace{(V_t^{\text{cl}} - V_{t-1}^{\text{cl}})}_{\text{close-to-close diff}} \times 100$$

$$\text{tc_cost}_t = |X_t - \text{held_pos}_t| \times TC_{\text{per unit}}, \qquad TC_{\text{per unit}} = 2 \text{ legs} \times \$0.02 \text{ half-spread} \times 100 = \$4.00$$

6.4 P&L Formula — Strike-Change Bars

$$\text{straddle_pnl}_t = X_t \times \text{vol_scale}_t \times (V_t^{\text{cl}} - V_t^{\text{op}}) \times 100$$

$$\text{tc_cost}_t = \big(|\text{held_pos}_t| + |X_t|\big) \times TC_{\text{per unit}} \qquad \text{(close old + open new)}$$

The gap between the old straddle's final close and the new straddle's open is unobservable in our data and set to zero (conservative assumption).

6.5 Trading Table (Mapping to Requested Columns)

In the submitted test script (`lstm_straddle_test.py`), the P&L calculation is **raw** (no volatility scaling $\sigma^{\text{tgt}}/\sigma_t$) and uses **VWAP execution prices** with the bid-ask spread embedded. Each trade produces *two rows*—one per option leg (call and put).

Column definitions:

- **timestamp**: The bar at which the position change occurs, formatted as `"YYYY-MM-DD HH:MM"` in Eastern Time.
- **asset**: The specific option leg, e.g. `Call K=384` or `Put K=384`.
- **position**: The fractional change in raw (un-scaled) position. Positive = buy; negative = sell. For example, if the signal moves from 0 to +0.70, both the call and put rows show position = +0.70.
- **vwap_price**: The option-leg VWAP for the bar (mean of 1-minute VWAPs within the 15-minute window).
- **cashflow**: Cash received or paid for that leg, incorporating the \$0.02 half-spread:

$$\text{cashflow} = -\,\Delta\text{pos} \;\times\; \text{exec_price} \;\times\; 100$$

  where:
  - exec_price = vwap + \$0.02 when buying (Δpos > 0),
  - exec_price = max(vwap − \$0.02, 0) when selling (Δpos < 0).

  Negative cashflow = cash out; positive = cash in.
- **cum_pnl**: Running sum of all cashflows up to and including the current row.

Strike-change handling. When the ATM strike shifts between bars, the old position is *closed* (two rows at the old strike) and a new position *opened* (two rows at the new strike), all at the same timestamp.

Example. Assume $K = 384$ straddle on Jan 3 and $K = 385$ on Jan 4. The LSTM fires $X_t = +0.70$ at Jan 3 10:00 (closed by EOD at 10:15), and $X_t = +0.55$ at Jan 4 10:00 (closed by EOD at 10:15). Half-spread = \$0.02.

Timestamp	Asset	position	vwap_price	cashflow	cum_pnl
Jan 3 10:00	Call K=384	+0.70	\$2.50	−\$176.40	−\$176.40
Jan 3 10:00	Put K=384	+0.70	\$2.47	−\$174.30	−\$350.70
Jan 3 10:15	Call K=384	−0.70	\$2.40	+\$166.60	−\$184.10
Jan 3 10:15	Put K=384	−0.70	\$2.55	+\$177.10	−\$7.00
Jan 4 10:00	Call K=385	+0.55	\$2.80	−\$155.10	−\$162.10
Jan 4 10:00	Put K=385	+0.55	\$2.76	−\$152.90	−\$315.00
Jan 4 10:15	Call K=385	−0.55	\$2.75	+\$150.15	−\$164.85
Jan 4 10:15	Put K=385	−0.55	\$2.72	+\$148.50	−\$16.35

Bars with no position change (e.g. Jan 3 09:45, Jan 4 09:45 where the signal is forced to 0 and held_pos is already 0) produce no rows in the cashflow table.

6.6 Step-by-Step Walkthrough (Based Solely on the Trading Table Above)

The following verifies two representative trades directly from the table in Section 6.5.

Rows 1–2 — Jan 3 10:00: entering a position (buy straddle). At the close of bar 09:45 the LSTM emits signal $X_t = +0.70$. The held position is 0, so Δpos = +0.70 (buying). Two rows are generated:

- **Call K=384**: VWAP = \$2.50, exec = 2.50 + 0.02 = \$2.52.

$$\text{cashflow} = -(+0.70) \times \$2.52 \times 100 = -\$176.40$$

- **Put K=384**: VWAP = \$2.47, exec = 2.47 + 0.02 = \$2.49.

$$\text{cashflow} = -(+0.70) \times \$2.49 \times 100 = -\$174.30$$

Running cum_pnl after these two rows: $-\$176.40 + (-\$174.30) = -\$350.70$. ✓

Rows 3–4 — Jan 3 10:15: closing the position (sell straddle). Last bar of Jan 3: the signal is forced to 0. The held position is +0.70, so $\Delta\text{pos} = -0.70$ (selling). Two rows are generated:

- **Call K=384**: VWAP = \$2.40, exec = 2.40 − 0.02 = \$2.38.

$$\text{cashflow} = -(-0.70) \times \$2.38 \times 100 = +\$166.60$$

- **Put K=384**: VWAP = \$2.55, exec = 2.55 − 0.02 = \$2.53.

$$\text{cashflow} = -(-0.70) \times \$2.53 \times 100 = +\$177.10$$

Running cum_pnl: $-\$350.70 + \$166.60 + \$177.10 = -\$7.00$. ✓

Round-trip decomposition (Jan 3 trade):

$$\begin{aligned}
\text{Raw price change} &= 0.70 \times \big[(2.40 - 2.50) + (2.55 - 2.47)\big] \times 100 = 0.70 \times (-\$2.00) = -\$1.40 \\
\text{Spread cost (4 legs)} &= 4 \times 0.70 \times \$0.02 \times 100 = \$5.60 \\
\text{Net P\&L} &= -\$1.40 - \$5.60 = -\$7.00 \quad ✓
\end{aligned}$$

6.7 Summary: Loss Function vs. Backtest P&L

Quantity	`compute_loss` (training)	`build_cashflow_table` (test)
Unit	Dimensionless log-return	Dollars (×100 multiplier)
Position held during t	X_{t-1} (via `prev_signal`)	`signal.shift(1)`
Return basis	r_t^{str} (log)	VWAP execution price difference
Vol scaling	$\sigma^{\text{tgt}}/\sigma_t$ (in return)	**None** (raw P&L)
TC basis	$c\,\sigma^{\text{tgt}}\,\lvert\Delta(\text{vol-scaled pos})\rvert$	Spread embedded: VWAP ± \$0.02
Output format	Single row per bar (straddle)	Two rows per trade (call + put legs)

The training loss uses volatility-scaled returns to optimise Sharpe ratio, while the submitted test script computes raw dollar P&L using VWAP execution prices with the bid-ask half-spread (\$0.02 per leg) embedded directly in the execution price.